Table of Contents

1. Introduction

1.1 Brief Overview of Internship

As part of my third-year Software Engineering program, I undertook a capstone internship at AMILI Pte Ltd, Southeast Asia's first precision gut microbiome company. During this internship, I was exposed to real-world data challenges within healthcare, particularly around donor data pipelines where ensuring high data quality is essential for research accuracy and operational efficiency.

I used these problems to motivate my capstone project, which is to create a versatile, machine learning-based data validation and correction tool for general use by various industries and workflows. My main focus was on healthcare data as it was the case of my AMILI placement; however, the solution was crafted with larger range in mind, addressing the common data quality issues concerning organisations from finance to education and e-commerce.

The report presents the development path from Milestone 1 to Milestone 2, that included 14 weeks of hard-core research, development and iteration. It also illustrates how the feedback influenced the project's direction, how the technical challenges resulted in architectural decisions and how the solution changed into an efficient, maintainable system.

1.2 Project Goal and Relevance

My project is centered around the creation of a machine learning-based data validation and correction system in order to resolve the problems caused by low quality data in survey datasets. The system is based on AMILI's need to handle a lot of donor survey data containing frequently occurring errors such as incorrect phone number format, age values that are not possible and the mixing up of email formats. The system is not going to be an exclusive solution for the company; instead, it will be a modular and extensible validation-and-suggestion tool with the first proof of concept being the implementation of the validation of healthcare phone numbers and blood sugar levels.

I could not use OpenAI's API due to the sensitivity of data and healthcare privacy regulations. I utilised a two-stage, human-in-the-loop design. The first step of the system is an automatic validation that picks out the records which are not following the expected patterns and then it comes up with ML-based correction suggestions for the users to accept or reject, thus ensuring accuracy and accountability in a controlled setting.

1.3 Key Improvements Since Milestone 1

After submitting Milestone 1 and receiving feedback from my workplace and academic supervisors, I made a considerable overhaul of the project, transforming it from an ambiguous concept through the application of a practical solution with clear limitations and realistic expectations. These changes filled all the major gaps pointed out in the review and showed improvement in project management, technical decision-making and stakeholder communication. The main improvements were changing the workflow diagram to depict full data flow, showing integration points, decision logic and user interactions and specifying the delivery method as a Streamlit web application instead of a REST API based on usability needs. Additionally, I made a simpler system architecture by cutting down the codebase to what is necessary for the core functionality.

Most importantly, I changed the definition of the project scope, going from a complicated system with characteristics like automatic field detection, pre-trained NLP models and a plugin architecture to a straightforward tool that had manual column selection, domain-

specific models trained on user data and a ML pipeline that was very effective and quick. The move from automated complexity to focused simplicity not only resulted in better quality of code, better performance and clearer value proposition but also allowed faster development, easier testing and a more maintainable long-term evolution.

2. Milestone 1 Feedback and Refinements

2.1 Summary of Milestone 1 Feedback

Throughout the Milestone 1 review, my academic and workplace mentors noted several major shortcomings in the project's documentation and planning which would have to be dealt with before the development could move on. Their critique exposed significant flaws in the project's scoping, scheduling and communication.

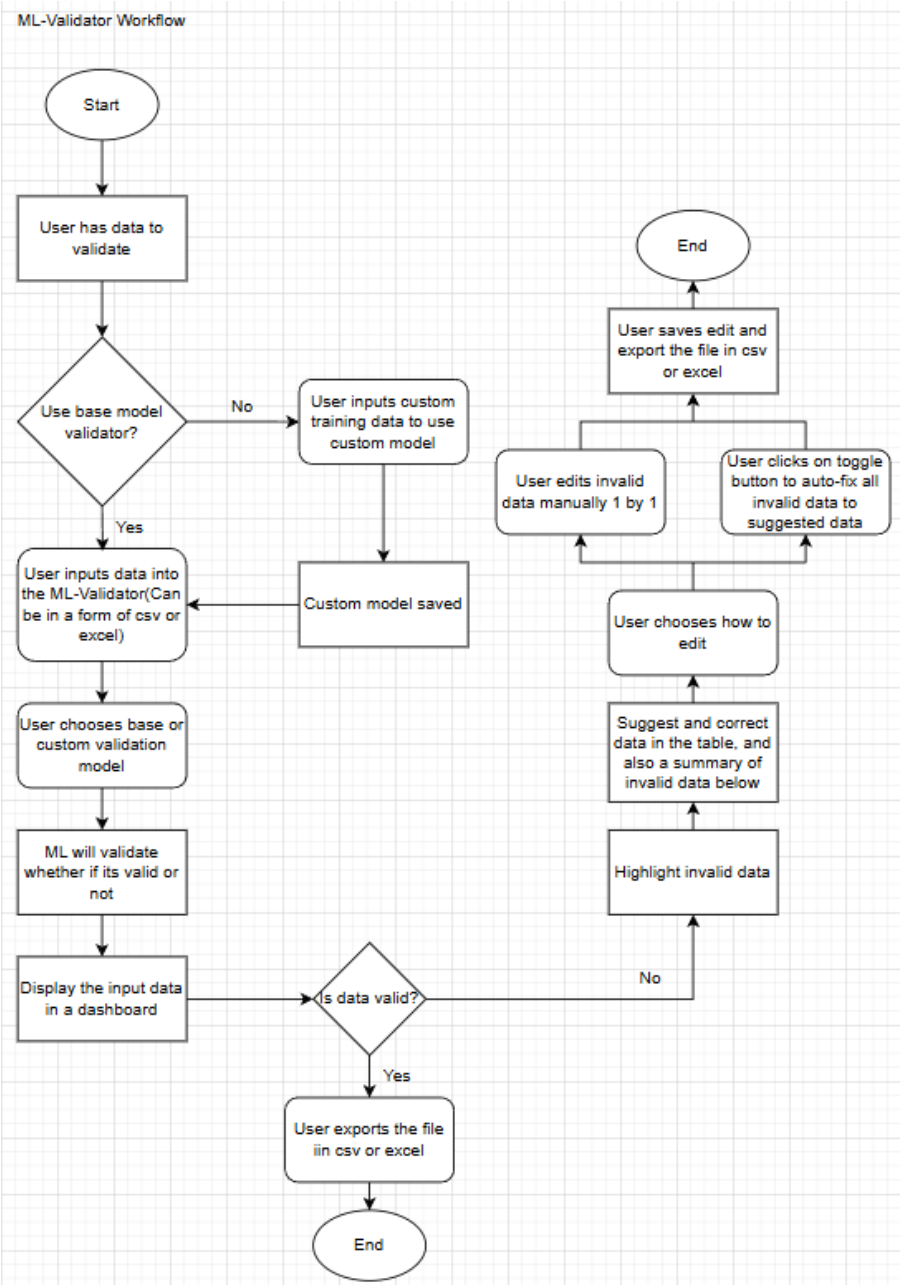
To begin with, the workflow diagram was described as too simple and lacking crucial architectural details, such as component interactions, the areas of application for the machine-learning models, user-system interaction and the decision logic that would control validation and correction. Another point was that the implementation strategy was not clear, leaving it up to the readers to decide whether the solution would be a REST API, a web app, or a combination of both, which in turn posed a challenge for planning integration, infrastructure and daily user interaction.

These gaps had become my top priority during Weeks 1-2 of Milestone 2, as their resolution was indispensable to alignment, setting realistic expectations and strong foundation for the rest of development to follow.

2.2 Improvements Implemented

2.2.1 Enhanced Workflow Diagram

In Milestone 1, my workflow diagram was overly linear and did not reflect the actual complexity of the data pipeline.



Screenshot 1: User Flow Diagram

The diagram has been redesigned in order to provide a more straightforward end to end view of the entire system, consisting of file uploading and final exporting. Users, at first, make up their minds on whether to go for the base validator, which is already aligned to standard healthcare-related columns, or to prepare a custom model by giving their own labelled training data in CSV or Excel file. Then, they upload their actual CSV or an Excel file into the ML Validator for validation. Base validators quickly map out the relevant columns, whereas custom validators need the user to map out their own columns before clicking the Validate button.

The validation process has ended; therefore, a table will be shown to the user by the UI with coloured-coded feedback. The valid entries will be shown in green and the invalid ones will be shown in red. At this time, the users can either export the file right away if everything is valid or deal with the red cells through two different editing modes. They can either manually correct the values in the table or utilise the auto-suggest feature that applies suggested fixes for each invalid row. The updated values will be highlighted in orange after being edited and the user will be able to export the cleansed dataset, thus completing the workflow.

2.2.2 Clarification of Delivery Method

The delivery and access of the solution were the main issues that caused all the uncertainties in Milestone 1. The first suggestion referred to access via an API, but it did not clarify if a REST API was meant, what the authentication process would be like, how users would interact with the system in their daily life and what the proposed infrastructure for the deployment would be.

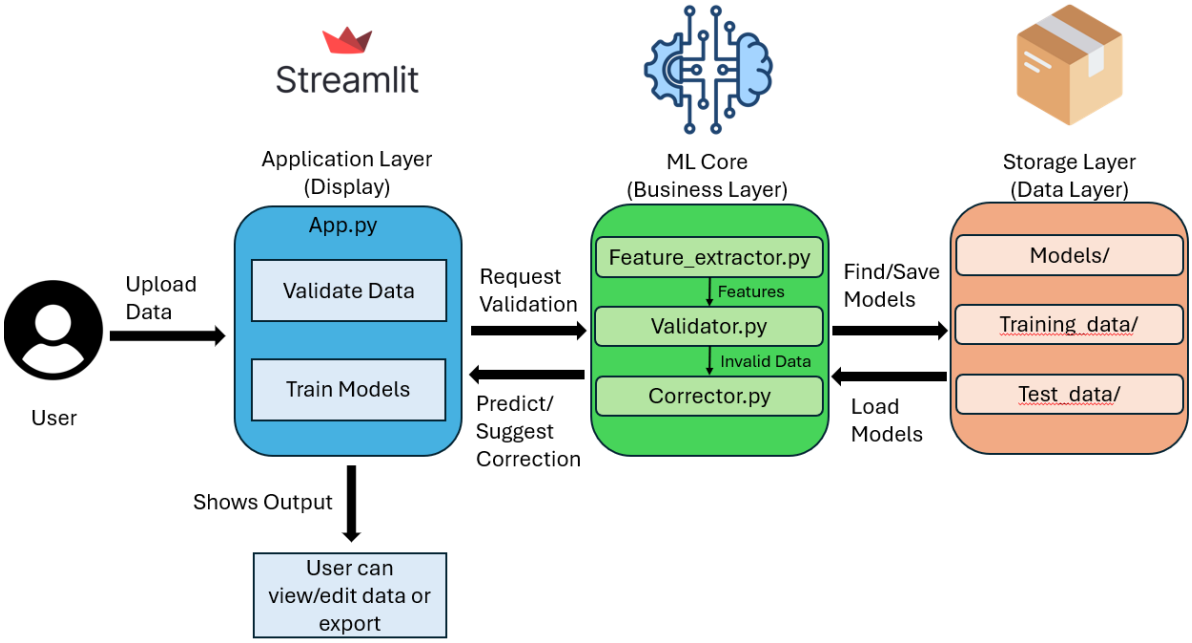
Discussions with my supervisor at work contributed to the decision of implementing the solution as a web app developed with Streamlit rather than going the REST API way. The decision took into account the non-technical end users who cannot code, oversee API authentication, or interpret JSON replies but find browser-based tools easy to use. The application will be hosted on a server in the final project phase, enabling users to interact with it via a common web browser by entering a special URL without any local installation or setup being necessary.

Many factors influenced this decision. Looking at the issue of accessibility, the staff from the laboratory, operations and data coordination had gotten accustomed to using spreadsheet-style dashboards, so a web interface was the most reasonable choice. The plan for server-based deployment will get rid of all the hassles such as users installing software on their computers, managing dependencies, or setting up development environments.

In addition, Streamlit facilitated rapid development cycles thus making it possible to test in a matter of minutes the alterations in the user interface, the addition of features and the changes in visualisations all without having distinct frontend and backend components.

3. System Design and Architecture

The ML-Data-Validator system incorporates a basic, single-layer structure with four main modules along with a Streamlit application. This structure divides the functions into three rational components: display, business logic and data management.



Screenshot 2: Architecture Diagram

The display layer constructed with Streamlit presents an interactive webpage containing two main tabs. In the Validate Data tab, users are permitted to upload either CSV or XLSX files, which are then automatically parsed and visualised in an editable grid.

In the business logic layer, the ml/ package contains the validation and correction engines. The validation engine generates 67 generic features from any text input, which consist of length metrics, ratios of character types, counts of special characters, positional features, pattern detection, n-gram analysis and numeric value features. A Logistic Regression classifier from scikit-learn with balanced class weights to deal with imbalanced datasets is thus fed with features. The correction engine uses Python's difflib SequenceMatcher for similarity-based matching instead of machine learning. It retains valid instances from training data and uses them for comparison of invalid inputs in terms of sequence similarity. If an input is deemed invalid, the corrector identifies the closest matching valid instance and proposes it as a correction.

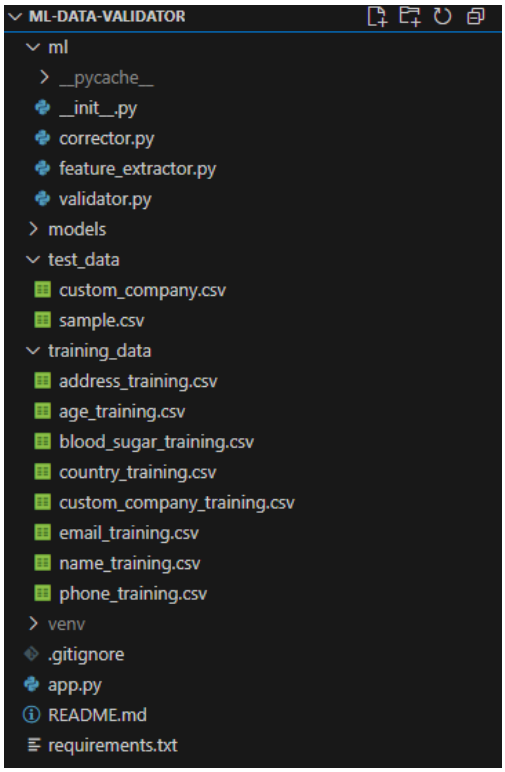
What's more, the data layer takes care of maintenance of training datasets and model persistence. The built-in system has eight datasets ready to be used, which together make up 2,047 labelled examples. The datasets are very helpful for supervised learning while preventing sensitive information from being disclosed. Model serialisation is performed using joblib for validators and pickle for correctors, which in turn, facilitates rapid loading and uninterrupted reuse of the models. The current system has a total of 18 pre-trained models.

4. Implementation Details

4.1 Project Structure and Code Organisation

The project is based on a flat, one-layer architecture that has five Python files. The main Streamlit app, app.py, is located in the root directory and it takes care of the UI workflow

including file upload, column mapping, validation, results display with colour-coded grids, correction suggestions and CSV export.



Screenshot 3: Project Structure

The ml/ package has three main components:
1. **validator.py** which implements the Logistic Regression classifier with training, batch validation and model persistence using joblib
2. **feature_extractor.py** which extracts 67 generic features from any text including length metrics, character ratios, special characters, positional features, pattern detection and numeric value features.
3. **corrector.py** which implements similarity-based correction using difflib SequenceMatcher.
The training logic is directly integrated into validator.py and corrector.py rather than being separated into different files.

The data structure contains training_data/ with eight CSV files (2,047 labelled examples), models/ with 18 pre-trained pickle files (9 validator/corrector pairs) and test_data/ with sample validation files. All the shared utilities are included within the three core modules. The requirements.txt file contains the core libraries (scikit-learn, pandas, streamlit) and the project root also has README.md and .gitignore.

This simple structure makes it easy to see the distinction between UI and business logic, has few files for easy navigation, integrates training and validation logic and has a lightweight design which makes it possible to have new validators simply by training new models with no need to restructure the code.

4.2 Synthetic Dataset Generation and Error Modelling

Because of the privacy policy which restricts the use of real patient data, I created artificial datasets that imitated the realistic error patterns that are seen in healthcare data entry.

The synthetic method has its downsides: it could fail to capture infrequent edge cases that were not foreseen during generation, the error frequency distribution is grounded on estimations rather than actual data and the model will need to be re-trained with real anonymised data to reach the target production performance that is optimum.

4.3 ML Model Evolution Summary

After evaluating multiple machine learning approaches, the validation component was implemented using scikit-learn's Logistic Regression with the following configuration:

- max_iter=1000 for sufficient convergence on the 67-dimensional feature space
- class_weight='balanced' to handle imbalanced datasets with unequal valid/invalid ratios
- random_state=42 for reproducibility across runs

Model	Accuracy	Training Time	Model Size	Complexity	Interpretability	GPU Required
Logistic Regression	89-98%	<1 sec	<2MB	Low	High	No
XGBoost	90-99%	2-5 sec	5-10MB	High	Medium	No
Neural Networks	90-99%	10-30 sec	10-50MB	Very High	Low	Yes
Pre-trained NLP (spaCy)	85-90%	N/A (pre-trained)	500MB-2GB	Very High	Very Low	Depends

Table 1: Validation Algorithm Comparison

As shown in Table 1, Logistic Regression was selected over alternatives such as XGBoost, Neural Networks and pre-trained NLP models (spaCy) due to its optimal balance of accuracy (89-98%), training speed (<1 second), model size (<2MB) and interpretability. This feature-based architecture enables validation of any data type through training on user-provided labelled examples, achieving comparable accuracy to more complex models while maintaining significantly lower computational overhead.

Algorithm	ML Training Required	Uses Training Data	Overhead	Deterministic	Complexity	Speed
difflib (Similarity)	No	Yes (as reference)	Zero	Yes	Low	Fast (<10ms)
Neural Seq2Seq	Yes	Yes	High	No	Very High	Slow (100ms+)
Gradient Boosting	Yes	Yes	Medium	Yes	High	Medium (50ms)
Transformer Models	Yes	Yes	Very High	No	Very High	Slow (200ms+)

Table 2: Correction Algorithm Comparison

For the correction component, a similarity-based matching approach was chosen over machine learning alternatives. The corrector utilises Python's difflib.SequenceMatcher to store valid examples from the training dataset as reference data. When invalid input is

detected, the algorithm calculates sequence similarity scores against all valid examples and suggests the most similar match if the similarity exceeds the configurable threshold of 60%. As demonstrated in Table 3, this approach eliminates the complexity, training overhead and unpredictability associated with neural sequence-to-sequence models, gradient boosting classifiers and transformer-based architectures. Unlike ML-based correction methods that require model training and produce non-deterministic results, difflib provides fast (<10ms), deterministic corrections using only the training data as reference.

The project concentrates on practical solutions that are easy to use over theoretical complexities by coining the phrase “train on your data” where not only the same simple pipeline but also supervised learning makes the club interval task transition easily.

5. Technical Challenges and Adaptations

5.1 Navigating Data Sensitivity and Privacy Constraints

The main obstacle was creating a validation system that can handle various types of data and at the same time not rely on strict rules set for certain formats. The task indeed called for the use of a general framework that is capable of learning rather than the employment of standard validators for phone numbers or other patterns of fixed nature like addresses.

I built the core feature extraction here, a pipeline that can analyse any text input and can do that through 67 different generic characteristics. Length metrics, character type ratios, special character frequencies, positional indicators and pattern detection are among the features analysed. It is the same validation architecture that treats different kinds of data such as telecommunication, healthcare records, e-commerce information, or custom business formats by simply training on different labelled examples. The system learns what "valid" means directly from the training data supplied by users.

The system is operating on the basis of only user-provided training data and is processing completely locally. Users are training their custom validators on 50-100 labelled examples without exposing the entire production datasets. All training, validation and correction of the model is done locally and the models are saved as .pkl files for future use. There are no external API calls or cloud dependencies whatsoever. This implies that all sensitive data stay in the user's environment and at the same time, accurate validation is applied that has been learned from the specific patterns of that user.

5.2 Identifying the Right ML Approach (Simplicity vs Complexity)

Selecting the appropriate machine learning approach for data validation required balancing accuracy, interpretability and maintainability. Rather than implementing cutting-edge techniques, I focused on proven methods that would deliver reliable results across diverse data types while remaining transparent and easy to debug.

I opted for Logistic Regression as the main validation algorithm because of its practicality in binary classification. It is a fast-training algorithm with a maximum of 2000 examples, it creates models not exceeding 2MB in size, it gives coefficients that are easy to understand, thus revealing the features that impact predictions and it is capable of giving good results if feature engineering is done properly.

The system uses a typical feature extraction pipeline that currently encompasses 67 different attributes such as length metrics, character type ratios, special character frequencies, positional patterns and domain-agnostic indicators. The same Logistic Regression model is used for validating phone numbers, emails, names, addresses, or custom business codes

through training on various labelled examples. The model is not based on hardcoded rules but rather learns validation patterns from the training data, which makes it very versatile and adaptable to any validation task without coding changes.

For corrections, I opted for a retrieval-based approach using Python's difflib.SequenceMatcher instead of generative models. This approach ensures that all the suggestions are real valid examples from the training set, it also eliminates poorly formed outputs and is able to run efficiently even with thousands of reference examples.

The architecture does not rely on heavy frameworks like PyTorch, XGBoost, or transformer models. The whole stack consists of scikit-learn for classification and Python built-in libraries for correction, leading to a virtual environment of 538MB as opposed to 2-3GB for deep learning frameworks. A 1,500-line codebase with 7 dependencies is much easier to maintain, understand, debug and modify than the ones with 20+ complex packages.

5.3 Architecture Simplification Decision

The old system was built around a 3-layer architecture that had more than 22 files, a plugin registry system, pre-trained NLP models (spaCy, transformers), automatic column detection and a total of 2-3GB dependencies.

After the analysis, it became evident that most of this architecture was of little value. The plugin registry made it possible for users to load validators dynamically, but this was not a necessity when users just have their data types and train models. Automatic column detection needed manual corrections in most cases and hence explicit user mapping was considered to be more reliable. Pre-trained NLP models were the ones that validators were based on and hence they needed the semantic understanding, but the pattern-based features proved sufficient. The three-layer separation made it possible that related logic was placed into multiple files without getting any benefit out of it.

The new system features Logistic Regression with generic feature extraction, user-driven column mapping, direct module imports and similarity-based correction. The number of dependencies was cut from more than 20 packages down to just 7 libraries, which also meant the removal of torch, xgboost, spacy and sentence-transformers. The size of the virtual environment was reduced from 2-3GB to 538MB. Codebase size went down from 3,000+ lines distributed among 22 files to 1,500 lines in 4 main files.

Debugging process was made easy because the clear execution paths were visible in less than 1,500 lines, as opposed to tracing through the 20+ interconnected modules. The simplified architecture has shown that deletion of unnecessary code can be the most impactful improvement rather than adding new features.

6. Competitive Analysis and Research

6.1 Project Update with Professor

After completing the major architectural simplification in Weeks 11–12, I met with my academic professor to review progress and plan the final phase for Milestone 2. The professor advised to look into competitive analysis. I needed to position ML-Data-Validator within the broader data validation landscape, comparing it to academic and open-source tools in terms of technical approach, usability, deployment and target audience, while highlighting gaps, differentiators and trade-offs.
Following this guidance, I designed a systematic research plan. I would survey academic systems via arXiv, ACM Digital Library and VLDB, focusing on machine learning or

automated data cleaning solutions. I would evaluate open-source libraries and applications to understand their strengths, weaknesses and design philosophies and review commercial data quality platforms to examine enterprise offerings and positioning. Finally, I would perform a structured gap analysis to identify where ML-Data-Validator meaningfully differs, clearly stating its advantages and limitations.

6.2 Review of Academic Solutions

After simplifying the architecture in Weeks 11–12, I reviewed academic research on data cleaning and validation, focusing on ML- or automation-based systems to situate ML-Data-Validator within the broader landscape and clarify design priorities.

HoloClean is one of a data cleaning system. It models cleaning as probabilistic inference using graphical models, incorporates functional and denial constraints and leverages weak supervision from external knowledge and data patterns. While general-purpose and academically sophisticated, HoloClean requires expertise in probabilistic modeling, substantial infrastructure, complex setup and lacks a user-friendly interface or production support. In contrast, ML-Data-Validator targets operations staff and analysts, requires minimal setup, validates files immediately through a web interface, focuses on phone numbers, runs on modest infrastructure and delivers rapid results. From HoloClean, I learned the value of extreme simplicity, avoiding formal constraint specification, favoring usability over maximal theoretical generality and focusing on a narrow, high-value use case.

ActiveClean is another data cleaning system which optimises data cleaning for ML model training using iterative, active learning-based prioritisation. It directs effort toward records that most affect model accuracy, but its scope is narrow, manual corrections are still required and the workflow assumes expertise in both ML and data cleaning. ML-Data-Validator differs by providing a standalone web application for general survey data validation, automatically generating correction suggestions and targeting non-technical users. Key lessons from ActiveClean include that automated corrections enhance user value, decoupling data quality tools from ML workflows broadens applicability and sophisticated academic techniques must be simplified for operational use.

6.3 Evaluation of Open-Source Tools

After the architectural simplification, I extended my competitive analysis to mature open-source tools like OpenRefine. OpenRefine is a long-standing desktop tool for cleaning and transforming messy data, offering a spreadsheet-like interface, clustering algorithms for standardisation, faceting for pattern discovery, reconciliation with external knowledge bases, GREL for complex transformations and a full undo/redo history. Its strengths are maturity, flexibility and community support, allowing non-programmers to perform advanced cleaning once the interface is learned.

However, OpenRefine has limitations for my target users. It requires installation and IT approval, has a steep learning curve for advanced GREL operations, lacks ML-powered suggestions and emphasizes transformation workflows rather than validation checks.
In comparison, ML-Data-Validator is a lightweight, browser-based web application requiring no installation, focused on phone number validation with ML-based error detection and correction suggestions. It delivers value within minutes and automates pattern recognition and corrections, unlike OpenRefine, which relies on manual rule creation.

This comparison reinforced key design lessons: focused, narrow tools can provide more practical value than broad, complex ones; integrated ML suggestions reduce manual effort

and differentiate the system from purely rule-based tools; and web-based deployment lowers adoption barriers relative to desktop software. ML-Data-Validator and OpenRefine are complementary: the former suits routine operational validation, while the latter supports complex, one-off transformation tasks.

6.4 Market Gap Analysis

After reviewing academic systems and open-source, I identified five gaps that ML-Data-Validator addresses:

6.4.1 Usability for non-technical users

Existing tools assume substantial technical expertise. HoloClean requires knowledge of probabilistic modeling and command-line workflows while ActiveClean expects familiarity with ML training and iterative cleaning. OpenRefine demands learning GREL expressions and advanced faceting. ML-Data-Validator provides a simple web workflow which is to upload, select column, view results, apply suggestions then simple export. There is no scripting or configuration, delivering immediate visual feedback for rapid adoption.

6.4.2 ML-powered correction suggestions

Current solutions offer limited intelligent correction. HoloClean needs detailed constraints to propose repairs, ActiveClean identifies high-impact records but leaves corrections to users and OpenRefine relies on manual rules or clustering. ML-Data-Validator automatically generates candidate fixes with confidence scores using character-level ML patterns for phone numbers, without requiring constraints or rule-writing.

6.4.3 Focused domain application

Most tools are general-purpose, often requiring heavy customisation for specific domains. ML-Data-Validator deliberately targets phone number validation, using features such as country code recognition, digit and formatting patterns and positional rules, achieving strong accuracy without per-organisation tuning.

6.4.4 Deployment simplicity

Academic systems are hard to deploy. For example, OpenRefine requires desktop installation and enterprise platforms involve complex integration. ML-Data-Validator runs as a web app on a single internal server or modest VM which requires no local installation.

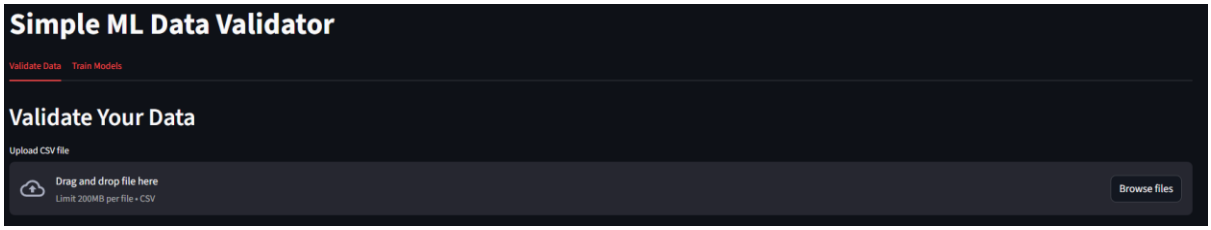
6.4.5 Privacy-conscious design

Cloud-based or external data transfer in existing tools conflicts with healthcare privacy requirements. ML-Data-Validator is built for on-premise, privacy-preserving use. All processing occurs internally, files are handled in memory, exports return directly to the user and models are trained on synthetic data, adhering to data minimisation, purpose limitation and controlled access principles.

7. Evaluation and Current Status

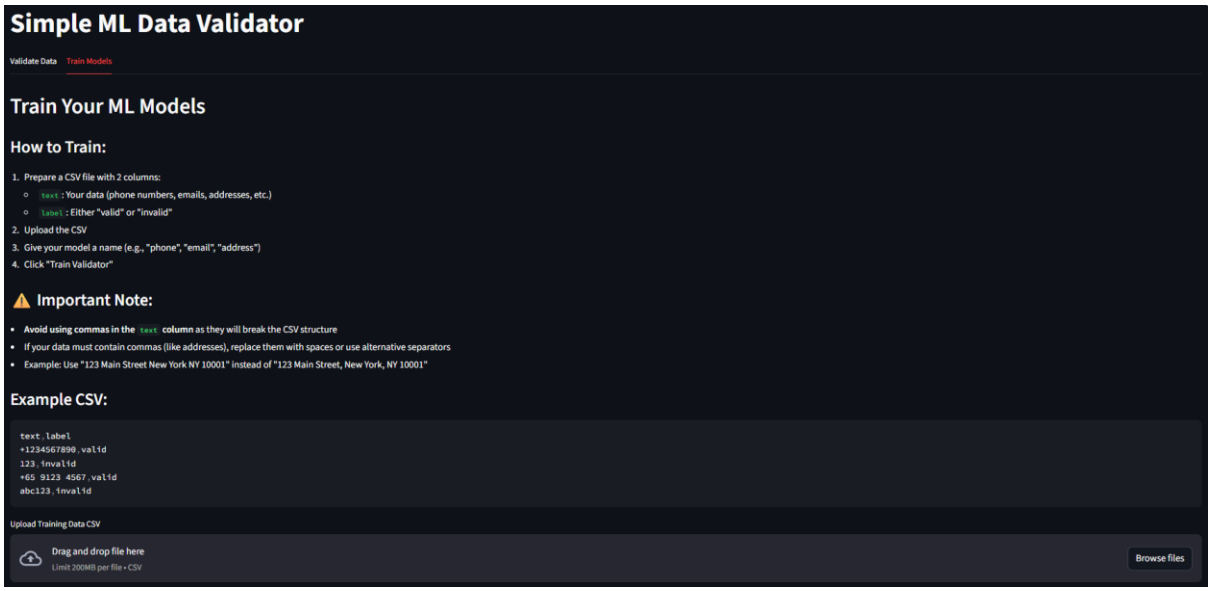
7.1 Completed Features and Functional Status

By Week 14, I delivered a functional ML-based data validation system with a Streamlit web interface, validation pipeline, correction engine and supporting documentation.



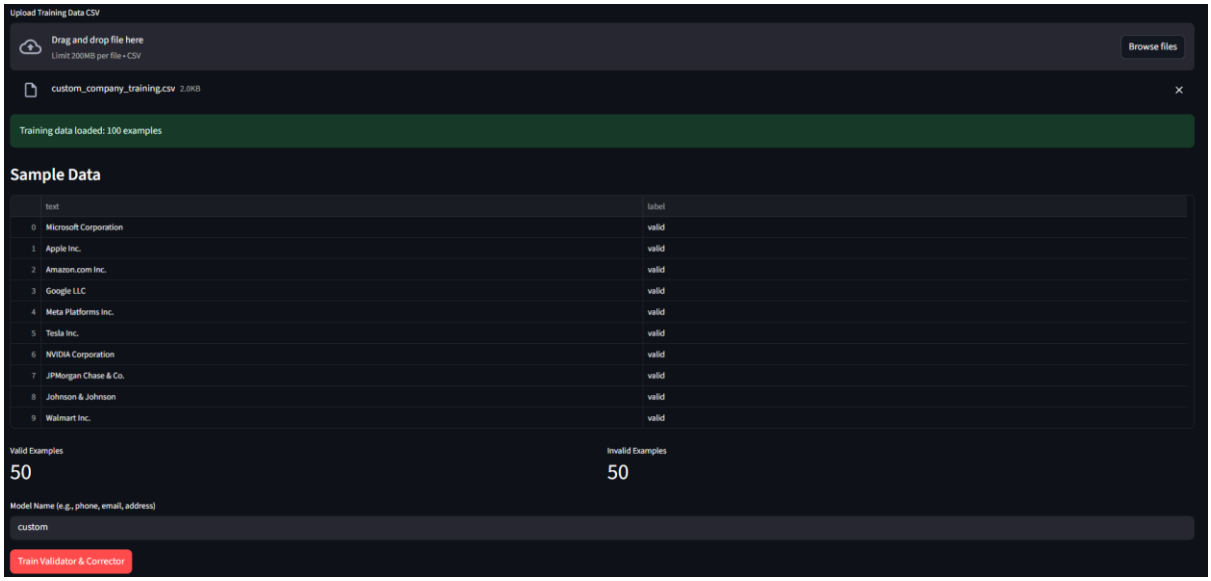
Screenshot 4: Validate Data and Train Models tabs

The Streamlit interface has two main tabs: "Validate Data" for handling CSV files and "Train Models" for making custom validators purposes.



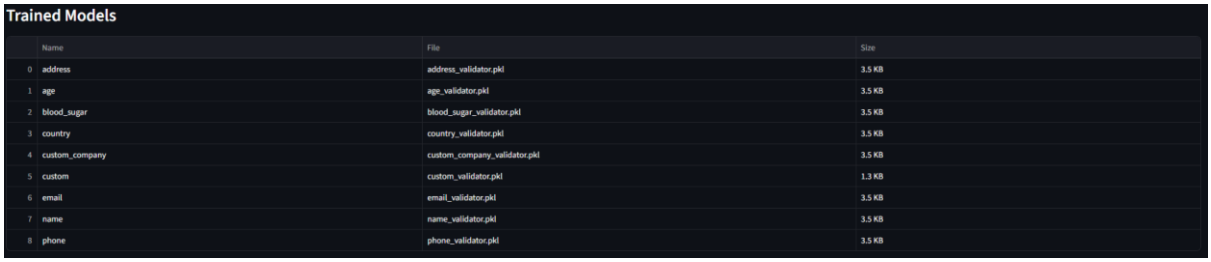
Screenshot 5: Train Model Instructions

The tab training consists of easy to follow instructions that guide users through the whole procedure: get a CSV file with text and label columns, upload the file, name the model and train the validator and corrector components.



Screenshot 6: Table of Uploaded Training Data

Users can see their uploaded training data in a table that presents sample entries with the corresponding validity labels. The interface also shows the training stats including the number of valid and invalid examples.

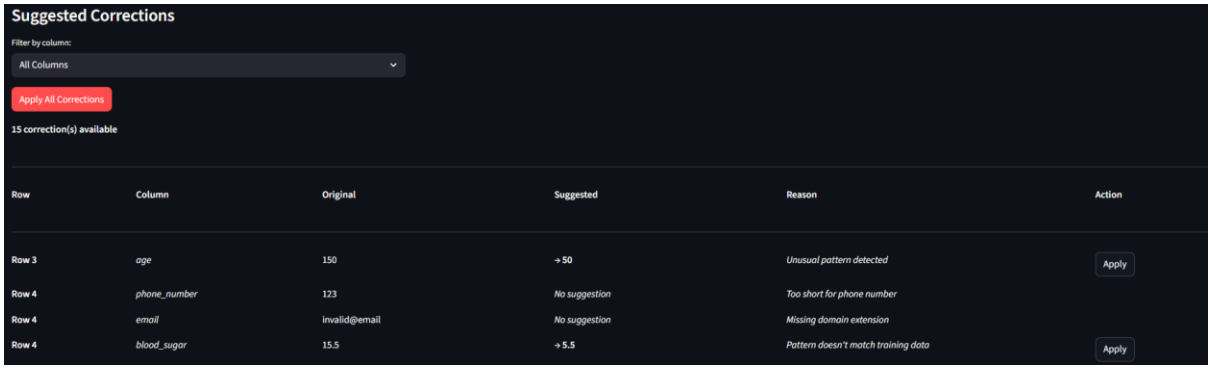


Screenshot 7: Existing Trained Models

Apart from that there is a section where all the existing trained models are listed with their respective file names and sizes, thus providing insight into the available validators.

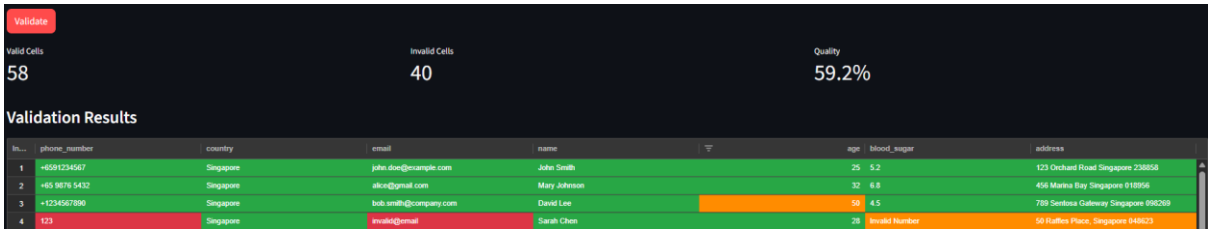
Screenshot 8: Preview table of uploaded data for validation

Going back to the Validation Data tab, the users will have to upload their CSV file first and then preview the data in a tabular format. Subsequently, after matching the columns to the appropriate validators, the system will process the data and show the results in an interactive color-coded grid: green cells will represent valid entries, red cells will indicate invalid data, and orange will highlight user-applied corrections.



Screenshot 9: Suggestion Table

Invalid entries will be shown in a suggestions table beneath, which will display the original value, the suggested correction, the reason for invalidity, and the individual "Apply" buttons. The users can either accept the suggestions one by one or apply all corrections in bulk before exporting the cleaned CSV.



Screenshot 10: Preview table with Applied Changes

This screenshot shows the aftermath of clicking the apply action button to edit the current data with suggestions from the model and also manual edits from the user. Edited values are indicated as orange to show that it has been changed.

7.2 Model and System Evaluation

The Logistic Regression model achieved high training accuracy on labelled data. The system processes validation requests quickly, with typical files validating in seconds on standard hardware. The correction engine uses similarity-based matching to suggest corrections for invalid entries. Users review suggested corrections before applying them, maintaining human oversight. End-to-end workflow of uploading, validating, correcting review and exporting is significantly faster than manual checking. The system provides immediate feedback on data quality issues, allowing staff to identify and address problems efficiently.

8. Current Status and Roadmap

8.1 Design Vision for Enhanced UI

The current UI supports a complete validation workflow. Several enhancements could improve usability for operations staff. The workflow could benefit from clearer step progression through Upload, Configure, Validate, Review and Export stages with visual indicators showing current position and allowing users to navigate between steps.

Validation results currently display in a colour-coded grid. Adding a summary panel showing valid/invalid counts and percentages would provide quick data quality assessment. Grouping similar errors together would help identify systematic issues. The correction interface shows

suggestions with reasons. A side-by-side comparison view highlighting differences between original and suggested values would clarify proposed changes. Keyboard shortcuts alongside mouse actions would speed up review workflows. Bulk operations for high-confidence corrections combined with manual review of uncertain cases would balance efficiency with accuracy.

Design improvements include consistent colour coding throughout the interface, clear visual hierarchy emphasizing important information, immediate feedback for user actions and responsive layouts supporting different screen sizes. These changes would reduce cognitive load and speed up common tasks for staff processing multiple files.

8.2 Upcoming Milestones

Future improvements could enhance both model performance and user experience. Model improvements include expanding training datasets to cover more edge cases and international formats, tuning Logistic Regression parameters for better generalisation and refining the similarity-based correction engine to handle more complex typo patterns. Adding contextual features to the correction engine could improve suggestion quality.

UI enhancements could provide better workflow guidance through clear step indicators, summary statistics showing validation results at a glance and improved correction presentation explaining why suggestions were generated. Interactive features like filtering invalid entries, sorting by confidence score and keyboard shortcuts would speed up repetitive tasks for power users. These improvements balance model accuracy, transparency and efficiency for operations staff handling regular validation workflows.

9. Application of Knowledge

9.1 Classroom Learning Applied to the Project

The internship allowed me to apply Software Engineering concepts to real-world problems. I implemented a machine learning-based data validation system using Logistic Regression for binary classification, achieving high training accuracy across multiple data types. The system uses comprehensive feature engineering, extracting 67 generic features from text data including length patterns, character type ratios (digit%, letter%, space%), special character counts, position features and n-gram patterns for typo detection.

Data processing relied heavily on Pandas for CSV file handling, filtering and data transformation. The feature extraction pipeline captures patterns without hardcoded rules, making it adaptable to phone numbers, emails, names, addresses and custom data types. Models are serialised using joblib for persistence and reuse.

Git workflows supported experimentation through feature branches, regular commits and clean reversions of unsuccessful approaches. The commit history shows the evolution from seq2seq experiments to the current similarity-based correction approach, demonstrating iterative development and willingness to pivot when needed.

Modular design and object-oriented principles structured the system. The architecture separates the Streamlit UI, ML logic and feature extraction into distinct components with clear interfaces like validator.validate returning tuples. This separation of concerns makes the codebase maintainable and extensible.

The system includes practical features such as batch validation for performance, confidence scoring to indicate prediction certainty, similarity-based correction suggestions using difflib's SequenceMatcher and an interactive UI with editable data grids and colour-coded validation

results. The project demonstrates end-to-end software development from data preparation to model training to deployment via a web interface.

9.2 Self-Learning and Use of External Tools

The internship required extensive self-directed learning beyond coursework. I deepened my Python expertise by learning Streamlit for web interfaces, advanced Pandas operations for CSV processing, scikit-learn's LogisticRegression with class balancing and joblib for model serialisation. I explored pattern matching through regular expressions and similarity algorithms using difflib's SequenceMatcher for correction suggestions.

My learning approach was systematic: study official documentation, build minimal working examples, test incrementally and consult community resources. I relied on Streamlit docs, scikit-learn guides, Pandas references and engineering blogs for practical ML patterns. Development tools included VS Code for coding and debugging, Git/GitHub for version control and branch management and Jupyter notebooks for prototyping feature extraction logic. The project history shows iterative refinement, including pivoting from seq2seq experiments to the current approach.

9.3 Practical Skills Development

The project strengthened my practical ML development skills across the full pipeline. The final system uses straightforward algorithms that work reliably rather than complex approaches that underperform. Technically, I developed model comparison skills by evaluating different approaches and choosing based on practical results. I improved code quality by simplifying architecture from 22 files to 4 core modules, documenting design decisions and creating clear component interfaces. The UI prioritises non-technical users with colour-coded validation, editable grids, correction suggestions and real-time feedback.

I practiced structured project management through Git workflows and clear documentation. Failed experiments like seq2seq became learning opportunities, teaching me when to pivot based on results.

The project reinforced practical lessons such as simplicity often outperforms complexity, working solutions beat perfect ones and tools must align with actual user workflows. I built a functional system that validates data across multiple types using one generic pipeline which demonstrated that well-designed simple solutions can be more valuable than sophisticated but brittle ones.

10. Conclusion and Reflections

10.1 Key Technical and Professional Learnings

The internship provided practical experience building an ML system from concept to deployment. Working without access to real data made synthetic data generation central to the approach, ensuring privacy compliance while producing reproducible training datasets that can be expanded over time. Exploring different approaches including seq2seq experiments developed practical judgment about solution complexity.

The experience reinforced that simpler models often outperform complex ones when properly designed. The final Logistic Regression implementation with comprehensive feature engineering proved more effective than earlier experimental approaches. Technically, the project strengthened Python ML skills including Pandas for data processing, scikit-learn for model training, Streamlit for UI development and Git for version control. Key engineering practices included feature extraction design of 67 generic features, modular architecture,

code simplification and comprehensive documentation. The work required translating user needs into technical features, designing validation workflows for non-technical staff and balancing automation with human oversight. Regular communication with supervisors through reports and demonstrations strengthened the ability to explain technical decisions clearly.

The internship developed practical judgment about trade-offs between solution complexity and effectiveness, when to pivot based on results and how to balance technical quality with delivery timelines. This experience established a foundation for building maintainable ML systems where usability and operational constraints matter as much as technical implementation.

10.2 Impact on My Engineering Skills

Building a complete ML validation system required balancing multiple concerns including model performance, user experience, maintainability and workflow integration. Technical correctness alone is insufficient if users cannot adopt the tool effectively. Key trade-offs shaped the final design. Logistic Regression provided interpretability and reliability over potentially marginal gains from complex models. Human review of corrections prioritised safety over full automation. The architecture supports phone, email, name and address validation through generic feature engineering rather than type-specific implementations.

The architectural simplification from 22 files to 4 core modules was the most significant learning experience. Initially, complexity felt like sophistication. The refactoring demonstrated that focused, maintainable solutions often deliver more practical value than comprehensive but convoluted ones. Well-designed simple systems are easier to understand and extend.

This experience reinforced that technology serves users. The system targets non-technical operations staff, so clarity and usability matter more than technical sophistication. A working Logistic Regression model with an intuitive interface provides more value than experimental approaches with poor usability. Delivering functional solutions that fit existing workflows outweighs optimising unused features. The shift from valuing complexity to valuing simplicity represents significant professional growth and will guide future engineering decisions.

10.3 Personal Growth and Development

The internship developed practical engineering judgment through direct experience with trade-offs and failed experiments. The seq2seq exploration initially seemed like wasted effort but taught valuable lessons about matching solution complexity to problem requirements. This experience clarified why simpler approaches work better for structured data validation and reinforced the importance of pivoting based on results rather than persisting with underperforming approaches.

Early in the project, complexity felt like competence. The initial architecture had over 20 files with elaborate abstractions. The refactoring to 4 core modules demonstrated that simplicity often requires deeper understanding than complexity. Well-designed simple systems serve users better, particularly non-technical staff who need reliable, straightforward tools. Maintainability benefits both users and future developers. Working on a validation system for research data highlighted the importance of understanding domain constraints. Requirements like privacy considerations and human oversight directly influenced architectural decisions including synthetic data generation and suggestion-based corrections rather than automatic changes.

The experience clarified career interests in building practical ML tools that augment human decision-making, designing accessible interfaces for technical systems and focusing on

maintainable solutions to real problems. The most significant takeaway is that effective engineering means building the right solution for actual user needs rather than the most technically sophisticated one.

11. References

1. Lwakatare, L. E., Rånge, E., Crnkovic, I., & Bosch, J. (2021). *On the experiences of adopting automated data validation in an industrial machine learning project.* arXiv preprint arXiv:2103.04095. https://arxiv.org/abs/2103.04095
2. Zhou, Y., Tu, F., Sha, K., Ding, J., & Chen, H. (2024). *A survey on data quality dimensions and tools for machine learning.* arXiv preprint arXiv:2406.19614. https://arxiv.org/abs/2406.19614
3. Streamlit Inc. (n.d.). *Streamlit documentation.* https://docs.streamlit.io
4. Streamlit Inc. (n.d.). *Tutorials – Streamlit.* https://docs.streamlit.io/develop/tutorials
5. MoldStud. (n.d.). *Best practices for developers creating automated ML workflows.* https://moldstud.com/articles/p-best-practices-for-developers-creating-automated-ml-workflows
6. Rekatsinas, T., Chu, X., Ilyas, I. F., & Ré, C. (2017). HoloClean: Holistic data repairs with probabilistic inference. *Proceedings of the VLDB Endowment, 10*(11), 1190–1201.
7. Krishnan, S., Wang, J., Wu, E., Franklin, M. J., & Goldberg, K. (2016). ActiveClean: Interactive data cleaning for statistical modeling. *Proceedings of the VLDB Endowment, 9*(12), 948–959.
8. OpenRefine. (n.d.). *OpenRefine: A free, open source, powerful tool for working with messy data.* https://openrefine.org/
9. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.
10. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
11. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.
12. Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems, 27*, 3104–3112.
13. Personal Data Protection Commission Singapore. (n.d.). *Overview of PDPA.* https://www.pdpc.gov.sg/overview-of-pdpa
14. Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. *IEEE Data Engineering Bulletin, 23*(4), 3–13.

END OF DOCUMENT